

Prueba Técnica — Consigna

El Bodegón · Sistema de reservas con chat IA

Desarrollador Backend — Supabase & Edge Functions

El **Bodegón** es un restaurante que quiere digitalizar la gestión de sus reservas y sumar un asistente con IA para que sus clientes puedan reservar conversando en lenguaje natural. El frontend ya está construido; tu tarea es desarrollar todo el backend que lo hace funcionar.

Tu objetivo

Desarrollar el backend en **Supabase** para: (1) **registro e inicio de sesión** con Supabase Auth, (2) un **sistema de reservas** disponible sólo para usuarios logueados, y (3) un **chat con IA (Google Gemini)** que permita crear reservas conversando, accesible también sólo con sesión iniciada.

1. El frontend que recibís

Te entregamos un frontend en React ya desarrollado (es un **mockup funcional**), con las pantallas de registro, login, reservas y chat listas. Está cableado para consumir tu backend: define los nombres de las funciones/endpoints y la forma de los datos que espera.

Podés verlo acá: <https://mockup-prueba-magne-7-26.vercel.app/>

- **No hace falta que modifiques el frontend**, salvo la configuración de conexión (variables de entorno de Supabase).
- **Revisá cómo el frontend llama al backend**: ese es el contrato que tenés que respetar (nombres, payloads de entrada y forma de las respuestas).
- El **schema de la base de datos** te lo entregamos junto con esta consigna. No tenés que diseñarlo desde cero.

2. Qué tenés que desarrollar

2.1 Autenticación (Supabase Auth)

- Registro de nuevos usuarios con email y contraseña.
- Inicio y cierre de sesión, con la sesión persistida para que el frontend refleje el estado del usuario.
- Creación del perfil del usuario asociado a su cuenta al registrarse.

2.2 Sistema de reservas

- Un usuario logueado puede **crear una reserva** (fecha, horario, cantidad de comensales y demás campos según el schema).
- Puede **consultar sus reservas** y **cancelarlas**.
- **Sólo un usuario logueado puede reservar**.
- Cada usuario sólo puede ver y operar sobre sus propias reservas.
- Validaciones básicas de negocio: fecha/horario válidos, cantidad de comensales razonable, etc.

2.3 Chat con IA (Google Gemini)

- El chat permite **crear reservas conversando en lenguaje natural** (por ejemplo: “quiero una mesa para 4 el viernes a las 21”).
- Implementá una **Edge Function** que reciba el mensaje del usuario, lo procese con Gemini, interprete la intención y los datos de la reserva, y cree la reserva confirmándola en la conversación.
- **El chat es sólo para usuarios logueados**. Si un usuario sin sesión intenta usarlo, debe recibir una respuesta indicándole que se registre o inicie sesión antes de poder reservar.
- **La reserva creada desde el chat debe quedar registrada igual que una hecha desde el formulario**, y respetar las mismas reglas.

3. Consideraciones técnicas

- Plataforma: **Supabase** — Auth, PostgreSQL y Edge Functions (Deno + TypeScript).
 - La **API key de Google Gemini** te la proporcionamos nosotros.
 - Respetá el contrato de datos que define el frontend para que la integración funcione de punta a punta.
-

4. Checklist de lo que esperamos ver funcionando

- Un usuario nuevo puede registrarse e iniciar sesión.
 - Un usuario logueado puede crear, ver y cancelar reservas desde el formulario.
 - Un usuario logueado puede crear una reserva conversando con el chat de IA.
 - Un usuario sin sesión que intenta reservar (formulario o chat) es correctamente bloqueado e invitado a registrarse.
 - Un usuario no puede ver ni tocar las reservas de otro.
-

5. Opcional (si te sobra tiempo)

No es obligatorio, pero suma si lo abordás o al menos lo mencionás como lo pensarías:

- Control de disponibilidad / cupos, para que no se pueda sobre-reservar un mismo horario.

Recordá

Es una prueba en vivo: podés hacer consultas, buscar documentación y pensar en voz alta. Preferimos un backend bien pensado, seguro y consistente antes que uno completo pero frágil. Si dejás algo afuera, contanos cómo lo resolverías. ¡Éxitos!